GOOGLE ANTIGRAVITY FIELD GUIDE

Antigravity Engineering & QA Playbook

Complete Operational Guide to Prompting, Pre-Flight Verification, and Safe Production Delivery in Multi-Agent Collaborative Teams

Project:	**ReportSaathi** (Plain-Language Medical Report Explainer)
Assigned Role:	**Person 3: Quality & Engineering Lead**
Collaborators:	**Person 1 (Product / Design Lead), Person 2 (DevOps / Systems Lead)**
Primary Environment:	**Google Antigravity IDE (Windows, PowerShell, Standard Sandbox)**
Repository:	`https://github.com/abhinavjanawade2242-dev/ReportSaathi-Project-Resources`
Author / Lead:	**Abhinav Janawade (Person 3)**
Version & Date:	**Version 1.0 — November 2026**

Document Objective

This document serves as an exhaustive, battle-tested manual for completing complex software engineering, quality assurance, and clinical safety tasks using **Google Antigravity**. It documents the exact prompts executed during the project, the rigorous pre-flight checks required before submitting each prompt, the strategic architectural rationale behind every command, and the troubleshooting nuances of Antigravity's sandboxed runtime.

Executive Summary of Completed Deliverables

Workstream	Deliverables Completed	Key Verification Metric
Deterministic Safety	`backend/safety/rules.py`, `banned_phrases.py`	100% deterministic emergency alerts prior to AI; 13 safety tests passing
OCR Pipeline	`backend/integrations/ocr.py`	Tesseract integration, uncertainty scoring & user correction re-validation
Privacy & Ephemeral Storage	`backend/safety/privacy.py`	Log scrubbing for PHI, path-traversal blocking, automatic TTL purge
API Contracts & Testing	`tests/test_api_contract.py`, `tests/test_privacy.py`	40 / 40 unit and contract tests passing (100% pass rate in 0.039s)
Frontend Web Application	`frontend/` (React 18 + Vite 5 + TypeScript)	6 core screens, offline status guard, empty state handling; builds in 770ms
Backend API Server	`backend/main.py` (FastAPI)	Live routes for upload, extraction, correction, explain, questions & feedback
CI/CD & Clinical Governance	`.github/workflows/ci.yml`, `06d-clinical-review-protocol.md`	Automated GitHub Actions CI; formal physician sign-off audit protocol

1. The Antigravity Operating Model

Google Antigravity is an agentic AI development environment capable of inspecting workspaces, executing shell commands, authoring multi-file modules, and scheduling reactive background tasks. Achieving flawless results requires understanding its architectural guardrails:

1.1 Sandbox Isolation: Standard Mode vs. Bypass Mode

- **Standard Sandbox Mode (`BypassSandbox: false`):** By default, Antigravity commands run in an isolated environment with zero outbound network access. Local compilation (`npm run build`), unit tests (`python -m unittest`), and local file operations must always run sandboxed without asking for user elevation.
- **Bypass Sandbox Mode (`BypassSandbox: true`):** Disables network isolation and allows external connections (e.g., `git fetch origin`, `git push origin`, `npm install`). Antigravity rules dictate: *Always attempt sandboxed first; only elevate the single network-dependent step*.

1.2 The Golden Rule: Continuous Remote Git Synchronization

> **MANDATORY WORKFLOW DIRECTIVE:** In collaborative multi-developer hackathon teams, teammates push code independently. Before proceeding to any new step or drafting new code, Antigravity MUST execute `git fetch origin` and inspect `git log HEAD..origin/main` to avoid catastrophic merge collisions or redundant work.

1.3 Respecting Persona & Team Boundaries

Role	Team Member	Strict Boundaries & Owned Modules
Person 1	Shrinidhi (Product / Design)	Owns PRD scope, UI/UX screens in Google Stitch, `docs/01`, `04`, `07`, `09`.
Person 2	Chethan (DevOps / Systems)	Owns backend specs, Gemini API integration, `docs/02`, `05`, `06`, `10`.
Person 3 (You)	Abhinav (Quality / Engineering)	Owns test fixtures, safety engine, OCR parser, contract tests, frontend code, and release QA.

2. The Pre-Prompt Verification Framework

Before submitting any instruction to Antigravity, apply the **5-Point Pre-Flight Checklist** to guarantee agent alignment:

1. **Remote Git State:** Has another team member pushed changes to `main`? (Run `git fetch origin`).
2. **Role Scope Validation:** Does this task belong to Person 3, or will it intrude on Person 1 or 2's documented ownership?
3. **Environment Prerequisite Check:** Are required tools installed? (e.g., Tesseract binary at `C:\Program Files\Tesseract-OCR\tesseract.exe`, Node.js v20+, Python 3.12+).
4. **Medical Safety Grounding:** Does this prompt violate safety rules (e.g., attempting disease diagnosis, medication dosage modification, or inventing reference ranges)?
5. **Artifact Verification:** Do downstream files exist before asking the agent to test them?

3. Step-by-Step Prompt Playbook: From Setup to Delivery

Step 1: Workspace Initialization & Team Role Anchoring

WHAT TO CHECK BEFORE GIVING THIS PROMPT:

- Verify that your target repository exists and you have write permissions.
- Identify your local scratch workspace directory (e.g. `C:\Users\abhin\.gemini\antigravity\scratch\`).
- Have your team roles and assignments clearly defined from hackathon kickoff.

VERBATIM PROMPT TO GIVE ANTIGRAVITY:

```
We are building ReportSaathi (a medical report and prescription explainer) as a 3-person team:
- Person 1 (Shrinidhi): Product Strategy & Design Lead (frontend UX, docs/01, 04, 07, 09)
- Person 2 (Chethan): Systems & DevOps Lead (backend specs, Gemini AI, docs/02, 05, 06, 10)
- Person 3 (Me): Quality & Engineering Lead (test fixtures, safety rules, OCR integration, frontend implementation, QA)

Clone our repo https://github.com/abhinavjanawade2242-dev/ReportSaathi-Project-Resources.git into the scratch directory.
Inspect docs/01 to docs/10 and summarize our MVP scope and my assigned QA tasks.
```

WHY WE GAVE THIS PROMPT:

Anchors the agent's identity as Person 3, preventing it from overwriting Person 1's product documents or Person 2's backend specifications. Forces the agent to ground all subsequent work in the agreed architectural decisions.

Step 2: Mandatory GitHub Synchronization Routine

WHAT TO CHECK BEFORE GIVING THIS PROMPT:

- Network status: Antigravity needs elevated bypass to reach GitHub.
- Ensure local working tree is clean (no uncommitted conflicting edits).

VERBATIM PROMPT TO GIVE ANTIGRAVITY:

```
Before proceeding to any step, verify the contents of GitHub:
Run 'git fetch origin' (using BypassSandbox: true if network blocks).
Check if any teammates have pushed new commits with 'git log HEAD..origin/main --oneline'.
Verify our local git status and report what has been modified or added.
```

WHY WE GAVE THIS PROMPT:

Maintains multi-developer synchronization. Teammates Chethan and Shrinidhi pushed documentation commits frequently; running this prompt prevents diverged git branches and painful merge conflicts.

Step 3: Creating Stage 6 Anonymized Test Fixtures (TEST-01)

WHAT TO CHECK BEFORE GIVING THIS PROMPT:

- Read `docs/06-test-plan.md` and `docs/06b-safety-rules.md`.
- Ensure no real patient personal health information (PHI) is included.
- Confirm the 10 required clinical scenarios (normal, elevated lipids, low Hb with prescription, missing ranges, blurry OCR, emergency red flags, multilingual).

VERBATIM PROMPT TO GIVE ANTIGRAVITY:

```
Create 10 comprehensive, fully anonymized medical report test fixtures in tests/fixtures/ (01 to 10)
adhering to docs/06-test-plan.md:
1. Normal CBC
2. High lipid profile
3. Low hemoglobin with iron prescription
4. Missing printed reference ranges (triggering no_range_on_report)
5. Blurry OCR report (triggering requires_verification)
6. Multi-panel CBC+LFT+KFT batch
7. Scanned PDF text extraction
8. OCR challenges (Indian numbering commas, '< 0.5', 'Negative')
9. Trilingual benchmark (English, Hindi, Kannada across Simple, Standard, Detailed)
10. Critical red-flag emergency (Potassium >= 6.8 mmol/L, Platelets <= 18,000 /uL)
```

WHY WE GAVE THIS PROMPT:

Establishes Test-Driven Development (TDD). In safety-critical healthcare software, you must define the gold-standard test cases and edge-case boundaries before writing a single line of application code.

Step 4: Implementing Deterministic Safety Engine (SAFE-01, SAFE-04, SAFE-05)

WHAT TO CHECK BEFORE GIVING THIS PROMPT:

- Verify that red-flag thresholds match clinical standards: Potassium (≥6.2 / ≤2.8), Platelets (≤20,000), Hemoglobin (≤7.0), Glucose (≤50 / ≥350), Creatinine (≥5.0).
- Verify rule: Red flags must NEVER depend on AI/LLM availability.
- Verify rule: Banned phrases must block diagnosis, medication changes, and doctor bypass.

VERBATIM PROMPT TO GIVE ANTIGRAVITY:

```
Implement the deterministic medical safety engine in backend/safety/:
1. backend/safety/rules.py:
   - Biomarker status classifier using STRICTLY report reference ranges (never invent ranges; return
no_range_on_report if missing).
   - Deterministic red-flag emergency evaluator for Potassium, Platelets, Hemoglobin, Glucose,
Creatinine.
2. backend/safety/banned_phrases.py:
   - Regex filter scanning AI output to block diagnoses, medication dose changes, and physician bypass
advice with safe fallbacks.
3. Write automated unit tests in tests/test_safety.py and execute them using Python's built-in
unittest module.
```

WHY WE GAVE THIS PROMPT:

LLMs can hallucinate or fail during API outages. Deterministic safety rules in plain Python execute instantaneously and guarantee that a patient with life-threatening hyperkalemia or thrombocytopenia receives an immediate emergency alert, even if Gemini is down.

Step 5: Tesseract OCR Engine & User Verification Re-Validation (OCR-06 to OCR-09)

WHAT TO CHECK BEFORE GIVING THIS PROMPT:

- Check that Tesseract OCR executable exists on the system (`C:\Program Files\Tesseract-OCR\tesseract.exe`).
- Review OCR extraction requirements from `docs/06-test-plan.md` (TC-08, TC-09, TC-10).

VERBATIM PROMPT TO GIVE ANTIGRAVITY:

```
Implement the OCR extraction engine and user correction workflow in backend/integrations/ocr.py:
1. 'run_tesseract': Wrap Tesseract CLI execution with fallback to system PATH.
2. 'assess_text_clarity': Calculate noise ratio and set requires_verification=True if clarity is poor.
3. 'parse_biomarkers_from_text': Extract test name, value, unit, and printed range using regex.
4. 'apply_user_corrections': Accept corrected text, re-parse biomarkers, and re-evaluate red flags.
5. Create tests/test_ocr.py verifying clean extraction, blurred text flagging, and user correction re-
validation.
```

WHY WE GAVE THIS PROMPT:

Medical OCR is imperfect; character degradation (e.g. 13.5 read as 1?.5) can cause dangerous interpretations. By scoring clarity and re-evaluating corrected text, we enforce the human-in-the-loop requirement mandated by clinical safety.

Step 6: Automated API Contract Test Suite (TEST-14, API-01 to API-12)

WHAT TO CHECK BEFORE GIVING THIS PROMPT:

- Read `docs/06c-api-contracts.md` and `shared/types.ts` .
- Confirm expected HTTP status codes: 200 (OK), 400 (Empty upload), 413 (Over 10 MB), 415 (Unsupported format), 422 (Validation error).

VERBATIM PROMPT TO GIVE ANTIGRAVITY:

```
Create tests/test_api_contract.py implementing 13 automated contract tests verifying:
- GET /api/health schema
- POST /api/upload 10 MB limit (413), empty file (400), and format validation (415)
- POST /api/extract payload structure and uncertainty flags
- POST /api/extract/{id}/corrections non-empty text validation (422)
- POST /api/explain multi-language options (English, Hindi, Kannada) and reading levels
- Consistent standardized error responses (success, message, error_code)
- Preserving the exact mandatory disclaimer: 'This explains your report. It does not replace your
doctor.'
Execute the entire test suite and verify 100% pass rate.
```

WHY WE GAVE THIS PROMPT:

Contract testing ensures that frontend and backend developers can work completely in parallel without contract drift. It provides an executable specification for Person 2's backend.

Step 7: Privacy, PHI Log Sanitization & Ephemeral Lifecycle (SAFE-08, OCR-10, TEST-13)

WHAT TO CHECK BEFORE GIVING THIS PROMPT:

- Review `docs/06b-safety-rules.md` Section 9 (Privacy and Data Protection).
- Confirm rule: No patient medical values or report snippets in logs or analytics.
- Confirm rule: Temporary uploaded documents must be purged completely (no permanent storage in MVP).

VERBATIM PROMPT TO GIVE ANTIGRAVITY:

```
Implement privacy protection and ephemeral storage lifecycle in backend/safety/privacy.py:
1. 'sanitize_log_message' and 'PrivacyPreservingLogFilter': Automatically redact PHI, lab values, and report snippets from logs.
2. 'EphemeralStorageManager': Manage temporary session directories, block path traversal (../), provide 'cleanup_document' on demand, and 'cleanup_expired' using TTL.
3. Add tests/test_privacy.py covering log redaction, path traversal security, and TTL deletion. Run all tests.
```

WHY WE GAVE THIS PROMPT:

Healthcare compliance mandates data privacy by design. Ensuring medical reports never hit persistent application logs and files are strictly purged protects patient confidentiality and prevents HIPAA/DPDP violations.

Step 8: Frontend Application Scaffolding (React + Vite + TypeScript)

WHAT TO CHECK BEFORE GIVING THIS PROMPT:

- Check Node.js and npm versions (`node -v`, `npm -v`).
- Review Screen flows in `docs/04-design.md` (Language, Upload, Check Values, Results, Doctor Questions, Share).
- Ensure medical disclaimer is positioned prominently on every view.

VERBATIM PROMPT TO GIVE ANTIGRAVITY:

```
Scaffold the complete React 18 + Vite + TypeScript frontend in frontend/:
- Implement the 6 core screens: LanguageSelector, UploadView (10 MB validation), CheckValuesView (with manual correction & empty state guard for non-medical images), ResultsView (red-flag banners & biomarker cards), DoctorQuestionsView (3-6 questions), and ShareView (PDF download & WhatsApp link).
- Add live backend status indicator badge in header.
- Add mandatory disclaimer footer: 'This explains your report. It does not replace your doctor.'
- Verify production build with 'npm run build'.
```

WHY WE GAVE THIS PROMPT:

Delivers the user-facing MVP application for Person 1. Incorporating the empty state guard prevents random non-medical photos from displaying phantom lab values, and the disclaimer ensures medical liability compliance.

Step 9: GitHub Actions CI/CD Pipeline Automation (REL-04)

WHAT TO CHECK BEFORE GIVING THIS PROMPT:

- Verify that all 40 automated tests pass locally.
- Verify that `frontend/package-lock.json` is tracked in git.

VERBATIM PROMPT TO GIVE ANTIGRAVITY:

```
Configure GitHub Actions CI workflow in .github/workflows/ci.yml:
1. Trigger on push and pull_request to main.
2. Job 'python-tests': Run Python 3.12 with 'python -m unittest discover tests -v' (all 40 tests).
3. Job 'frontend-build': Run Node.js 20 with 'npm ci' and 'npm run build' in frontend/.
Commit and push to origin/main.
```

WHY WE GAVE THIS PROMPT:

Automated CI is a non-negotiable release gate. It ensures that any future commits by teammates immediately trigger automated regression testing on GitHub before deployment.

Step 10: Clinical Review & Medical Sign-Off Protocol (SAFE-12, M-02)

WHAT TO CHECK BEFORE GIVING THIS PROMPT:

- Review `docs/09-release.md` Section 8.2 & 14 (Release Acceptance Gates).
- Identify the clinical review requirement: Qualified physician (MBBS/MD) review of red flags and benchmark explanations.

VERBATIM PROMPT TO GIVE ANTIGRAVITY:

```
Create docs/06d-clinical-review-protocol.md to fulfill SAFE-12 and release gate M-02:
1. Emergency Red-Flag Threshold Table with clinical rationales for Potassium, Platelets, Hemoglobin, Glucose, Creatinine.
2. Review of the 10 anonymized benchmark fixtures across English, Hindi, and Kannada.
3. Formal Physician Review & Sign-Off Sheet template including Doctor Name, Registration Number, State Medical Council, and approval checkboxes.
```

WHY WE GAVE THIS PROMPT:

Software engineers cannot self-approve clinical safety thresholds. Providing a structured audit sheet allows a licensed physician to formally sign off on emergency triage levels before real patients use the tool.

Step 11: FastAPI Application Server Implementation (backend/main.py)

WHAT TO CHECK BEFORE GIVING THIS PROMPT:

- Verify that `fastapi` and `uvicorn` are installed in the Python environment.
- Verify that `backend/safety/rules.py`, `banned_phrases.py`, `privacy.py`, and `integrations/ocr.py` are present.

VERBATIM PROMPT TO GIVE ANTIGRAVITY:

```
Create backend/main.py implementing the full FastAPI server adhering to docs/06c-api-contracts.md:
- CORS middleware for React frontend.
- GET /api/health
- POST /api/upload (enforcing 10 MB limit and ephemeral storage)
- POST /api/extract (Tesseract OCR + clarity scoring)
- POST /api/extract/{id}/corrections (user verification re-parsing)
- POST /api/explain (deterministic red flags + educational text generation + banned phrase sanitization)
- POST /api/questions (3 to 6 doctor consultation questions)
- POST /api/feedback (rating submission)
Verify the server loads cleanly with zero import errors.
```

WHY WE GAVE THIS PROMPT:

Transitions ReportSaathi from a collection of specifications into a fully functional, live-running application. The frontend and backend can now interact end-to-end.

Step 12: Team Handover & WhatsApp Updates Generation

WHAT TO CHECK BEFORE GIVING THIS PROMPT:

- Verify all tests pass (`40/40 OK`).
- Verify all local commits are pushed to `origin/main`.

VERBATIM PROMPT TO GIVE ANTIGRAVITY:

```
Draft concise, professional handover messages for our team WhatsApp group:
1. Message to Chethan (Person 2 - DevOps/Backend): Explaining how to pull main, import backend safety modules, and run the 40 contract tests.
2. Message to Shrinidhi (Person 1 - Product/Design): Outlining QA pass metrics in docs/09-release.md and sharing the clinical review audit sheet (docs/06d-clinical-review-protocol.md) for doctor sign-off.
```

WHY WE GAVE THIS PROMPT:

Completes the collaborative loop. Clear, jargon-free handover messages ensure team members know exactly how to integrate the engineering deliverables into their remaining tasks.

4. Antigravity Operational Best Practices & Troubleshooting

Issue / Error Encountered	Root Cause in Antigravity	Proven Solution & Prevention
getaddrinfo() thread failed to start	Standard Sandbox Mode blocks outbound network by default.	Re-run the command with `BypassSandbox: true` and the exact same `toolSummary`.
Write-Progress console buffer error (0x5)	PowerShell `Expand-Archive` fails in headless terminal buffers on Windows.	Use Python standard library (`zipfile.ZipFile`) to extract archives cleanly.
Pytest connection timeout in sandbox	Pytest was not installed locally and pip install blocked in offline sandbox.	Use Python's built-in `unittest` module (`python -m unittest discover tests`). Zero external dependencies.
Unused variable compilation error (TS6133)	TypeScript strict mode rejects imported but unused interfaces in React components.	Run `npm run build` locally to catch TS errors before committing. Strip unused imports.
Phantom lab values on non-medical uploads	Frontend mock client fell back silently to `sample-report.json` on backend errors.	Implement active backend health probe and empty state guard (`values.length === 0`) in UI.

5. Master Prompt Cheat Sheet

Phase	Pre-Flight Check	Key Antigravity Instruction	Expected Deliverable
1. Sync	Local working tree clean	`git fetch origin && git log HEAD..origin/main`	Confirmed zero merge conflicts
2. Fixtures	Verify 10 clinical scenarios	`Create 10 test fixtures in tests/fixtures/ adhering to docs/06`	10 anonymized JSON fixtures
3. Safety	Review red-flag cutoffs	`Implement deterministic rules in backend/safety/rules.py`	Red flags + banned phrase filter
4. OCR	Check Tesseract executable	`Implement Tesseract wrapper & clarity scorer in backend/integrations/`	OCR parser with correction flow
5. Contract	Inspect docs/06c contracts	`Create tests/test_api_contract.py for all endpoints`	13 contract tests passing
6. Privacy	Review data protection rules	`Implement log scrubbing & ephemeral storage in backend/safety/privacy.py`	PHI redacted + auto TTL purge
7. Frontend	Verify npm / Node.js v20	`Scaffold 6 screens in frontend/ with React + Vite + TypeScript`	Clean build in <800ms
8. CI/CD	All tests passing locally	`Configure GitHub Actions in .github/workflows/ci.yml`	Green automated build on GitHub
9. Clinical	Review release gate M-02	`Create docs/06d-clinical-review-protocol.md with doctor sign-off sheet`	Physician audit protocol ready
10. Backend	Verify fastapi & uvicorn	`Create backend/main.py implementing all FastAPI contract endpoints`	Live runnable API server